

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN



BÀI TẬP NHÓM
DỰ ĐOÁN GIÁ NHÀ

Học phần: Học sâu

Mã học phần: 858048

Nhóm lớp: 01

Giảng viên hướng dẫn: TS. Đỗ Như Tài

Sinh viên thực hiện: Đặng Thanh Phương: 3123580036

Phạm Trung Hiếu: 3123580014

Nguyễn Trương Cao Sơn: 3123580040

Lớp: DDU1231

Học kỳ 1, năm học 2026–2027

Thành phố Hồ Chí Minh, tháng 9 năm 2026

Thành viên	Phản phụ trách vai trò và chi tiết công việc	Đầu ra
Đặng Thanh Phương	Khai phá & Tiền xử lý dữ liệu: - Khám phá dữ liệu, xử lý ngoại lệ và điền khuyết. - One-hot encoding và chuẩn hóa. - Chia tập Train/Val và chuyển đổi thành PyTorch Tensors. Báo cáo nội dung 1-3: - Hiểu bài toán - Khám phá - Tiền xử lý dữ liệu	Các biến Tensor: X_train_t, y_train_t, X_val_t, y_val_t, X_kaggle_t. Báo cáo CRISP-DM trên Google Docs.
Nguyễn Trương Cao Sơn (Nhóm trưởng)	Xây dựng & Tối ưu Mô hình: - Xây dựng MLP bằng PyTorch. - Huấn luyện, đánh giá (Log-RMSE, MAE, R2). - Tuning siêu tham số, lưu trọng số. - Phát triển và kết hợp thêm các mô hình khác nhau Báo cáo nội dung 4: Xây dựng mô hình - Xây dựng mô hình MLP - Huấn luyện mô hình - Tuning và thử nghiệm các mô hình bổ sung	Tập trọng số mô hình tối ưu: best_model.pt, best_tuned_model.pt và tập log tuning_results.csv. Báo cáo CRISP-DM trên Google Docs. Kiểm tra rà soát lại báo cáo
Phạm Trung Hiếu	Triển khai & Báo cáo: - Load mô hình, dự đoán trên tập Test của Kaggle. - Chuyển đổi ngược logarit (np.exp(m1)) về giá thực. - Tổng hợp mã nguồn và viết báo cáo chuẩn CRISP-DM. Báo cáo nội dung: - Bìa, Lời mở đầu, Cảm ơn, Mục Lục - 5-6: Evaluation và Deploy, Submit	Tập nộp bài: submission.csv; bảng đánh giá evaluation_metrics.csv ; Báo cáo CRISP-DM trên Google Docs.

LỜI MỞ ĐẦU

Trong thời đại dữ liệu bùng nổ hiện nay, việc ứng dụng các kỹ thuật học máy và học sâu để khai thác thông tin từ dữ liệu thực tế đã trở thành một yêu cầu thiết yếu đối với sinh viên ngành Khoa học Dữ liệu. Bài toán dự đoán giá nhà (*House Prices - Advanced Regression Techniques*) trên nền tảng Kaggle là một bài toán kinh điển, giúp người học tiếp cận toàn bộ quy trình xử lý một dự án học máy thực tế — từ hiểu bài toán, khám phá và tiền xử lý dữ liệu, xây dựng mô hình, cho đến đánh giá và triển khai kết quả.

Trong báo cáo này, nhóm chúng em sử dụng dữ liệu Ames Housing để dự đoán SalePrice dựa trên các thuộc tính của căn nhà. Mô hình học sâu cơ sở là MLP được xây dựng bằng PyTorch; bên cạnh đó, nhóm thử nghiệm XGBoost, CatBoost và các mô hình tuyến tính có regularization để so sánh và tìm kiếm các dự đoán có khả năng bổ sung lẫn nhau khi kết hợp. Quy trình được trình bày theo sáu giai đoạn CRISP-DM, từ xác định mục tiêu đến đánh giá và nộp kết quả trên Kaggle.

Do thời gian thực hiện có hạn cùng với kiến thức và kinh nghiệm còn nhiều hạn chế, báo cáo khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được sự góp ý, chỉ dẫn của Thầy/Cô để hoàn thiện hơn trong những dự án tiếp theo.

Nhóm chúng em xin chân thành cảm ơn.

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành đến Thầy Đỗ Như Tài - giảng viên học phần **Deep Learning** đã tận tình giảng dạy, truyền đạt những kiến thức nền tảng và nâng cao về mạng nơ-ron nhân tạo, cũng như hướng dẫn, hỗ trợ nhóm trong suốt quá trình thực hiện đồ án này.

Nhờ những kiến thức và kinh nghiệm được thầy trang bị trong quá trình học tập, nhóm chúng em đã có thể vận dụng để hoàn thành bài toán dự đoán giá nhà — từ khâu xử lý dữ liệu, xây dựng mô hình học sâu bằng PyTorch, đến đánh giá và triển khai kết quả trên nền tảng Kaggle.

Trong quá trình thực hiện, do giới hạn về kiến thức và thời gian nên báo cáo không tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được sự nhận xét và góp ý từ thầy để rút kinh nghiệm cho những đồ án về sau.

Nhóm chúng em xin chân thành cảm ơn!

MỤC LỤC

LỜI MỞ ĐẦU.....	3
LỜI CẢM ƠN.....	4
MỤC LỤC.....	5
1. HIỂU BÀI TOÁN (BUSINESS UNDERSTANDING).....	7
2. KHÁM PHÁ DỮ LIỆU (DATA UNDERSTANDING).....	7
2.1. Khảo sát phân bố của biến mục tiêu (SalePrice).....	10
2.2. Khảo sát tương quan và phát hiện ngoại lệ (Outliers).....	10
3. TIỀN XỬ LÝ DỮ LIỆU (DATA PREPARATION).....	11
3.1. Loại bỏ các điểm ngoại lệ (Outlier Removal).....	11
3.2. Đóng gói hàm tiền xử lý dữ liệu (preprocess_housing).....	12
3.3. Phân chia tập dữ liệu, Chuẩn hóa đặc trưng & Chuyển đổi PyTorch Tensors.....	16
4. XÂY DỰNG MÔ HÌNH (MODELING).....	19
4.1. Xây dựng mô hình MLP bằng PyTorch.....	19
4.1.1 Kiến trúc mô hình ban đầu.....	19
4.1.2 Kiến trúc mô hình sau Tuning.....	20
4.1.3 Hàm mất mát.....	20
4.1.4. Bộ tối ưu và cơ chế regularization.....	20
4.2. Huấn luyện mô hình.....	21
4.2.1. Quy trình huấn luyện.....	21
4.2.2. Theo dõi validation.....	22
4.2.3. Kết quả MLP baseline.....	22
4.3. Tuning mô hình MLP.....	23
4.3.1. Mục đích tuning.....	23
4.3.2. Thiết kế các cấu hình tuning.....	24
4.3.3. Cấu hình MLP được lựa chọn.....	25
4.4 Thử nghiệm các mô hình bổ sung.....	25
4.5 Xây dựng mô hình Ensemble.....	27
4.5.1 Cơ sở của Weighted ensemble.....	27
4.5.2. Từ MLP đến Ensemble ba mô hình.....	27
4.5.3. Bổ sung Lasso vào Ensemble.....	27
4.6. Kiểm tra độ ổn định bằng 5-Fold Out-of-Fold (OOF).....	28
4.6.1 Thiết lập OOF.....	28
4.6.2 Kết quả OOF.....	29
4.7. Thử nghiệm mô hình tuyến tính và hoàn thiện Champion.....	29
4.7.1. So sánh Ridge, Lasso và ElasticNet.....	29

4.7.2 Tinh chỉnh thông số Lasso.....	29
4.8. Tổng kết giai đoạn Modeling.....	30
5. ĐÁNH GIÁ MÔ HÌNH (EVALUATION).....	30
5.1. Mục đích và dữ liệu đánh giá.....	30
5.2. Các độ đo sử dụng.....	30
5.3. Kết quả đánh giá mô hình cuối.....	31
5.4. Nhận xét và giới hạn.....	31
6. TRIỂN KHAI VÀ NỘP BÀI (DEPLOYMENT).....	31
6.1. Dự đoán trên tập test.....	31
6.2. Tạo file submission.....	32
6.3. Nộp bài và diễn giải điểm Kaggle.....	32
KẾT LUẬN.....	32

1. HIỂU BÀI TOÁN (BUSINESS UNDERSTANDING)

Mục tiêu bài toán: Dự đoán giá bán nhà tại thành phố Ames, bang Iowa , dựa trên 79 biến giải thích mô tả gần như mọi khía cạnh của một căn nhà ,từ diện tích, chất lượng vật liệu, năm xây dựng, đến vị trí khu dân cư.

Nguồn dữ liệu: Bộ dữ liệu Ames Housing do Dean De Cock biên soạn, được công bố trên Kaggle trong cuộc thi "*House Prices - Advanced Regression Techniques*",

Phân loại các biến đầu vào: ta chia thành 4 nhóm:

Nhóm	Ý nghĩa	Ví dụ
Liên tục	Giá trị số, đo lường liên tục	LotArea, GrLivArea, TotalBsmntSF, GarageArea
Rời rạc	Giá trị số, đếm được	BedroomAbvGr, GarageCars, FullBath, TotRmsAbvGrd
Định danh	Phân loại, không có thứ tự	MSZoning, Neighborhood, HouseStyle, SaleType
Thứ bậc	Phân loại, có thứ tự chất lượng	ExterQual, KitchenQual, BsmntQual (thang Ex > Gd > TA > Fa > Po)

Biến mục tiêu: SalePrice — giá trị số liên tục (USD).

Tiêu chí đánh giá của cuộc thi: RMSE trên logarit của SalePrice

2. KHÁM PHÁ DỮ LIỆU (DATA UNDERSTANDING)

Kích thước dữ liệu ban đầu:

- Tập train: 1460 dòng × 81 cột (bao gồm Id và SalePrice)
- Tập test: 1459 dòng × 80 cột (không có SalePrice, cần dự đoán)

Số dòng train: 1460 | Số cột train: 81
Số dòng test : 1459 | Số cột test : 80

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType	SaleCondition	SalePrice
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2008	WD	Normal	208500
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	2007	WD	Normal	181500
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9	2008	WD	Normal	223500
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2006	WD	Abnorml	140000
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	12	2008	WD	Normal	250000

5 rows x 81 columns

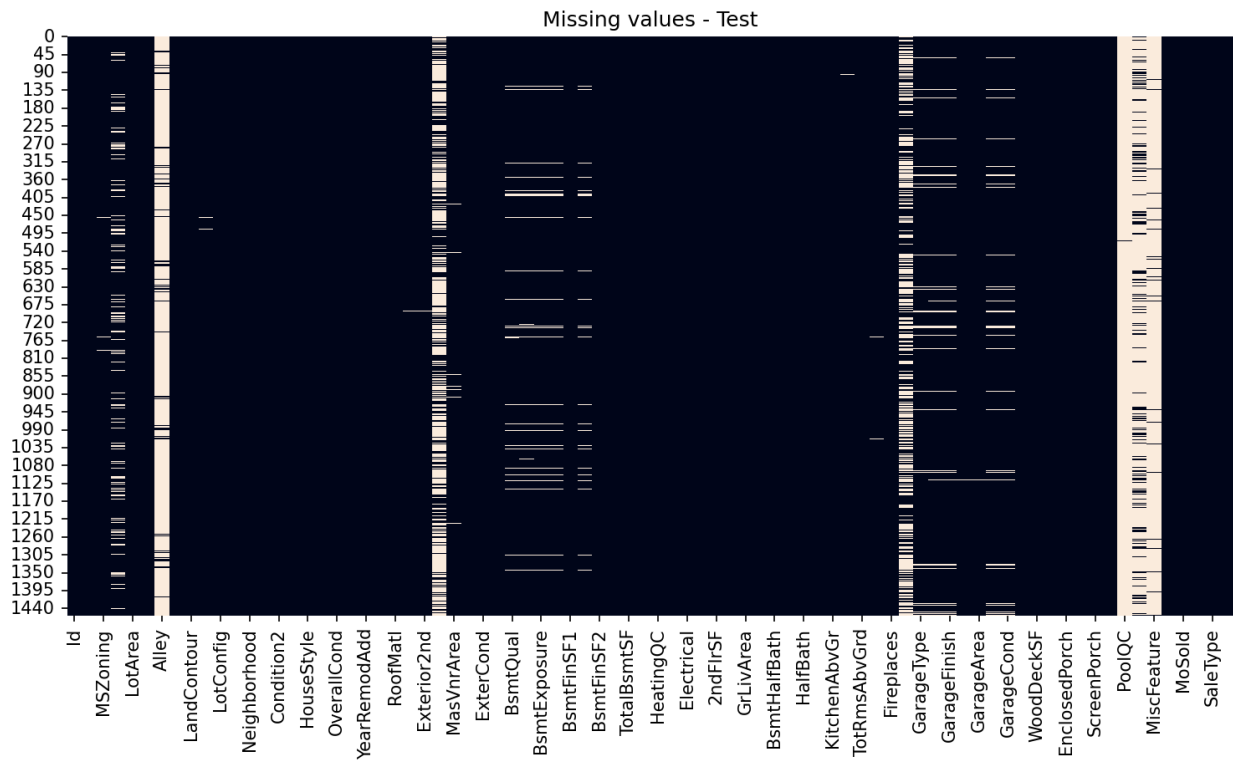
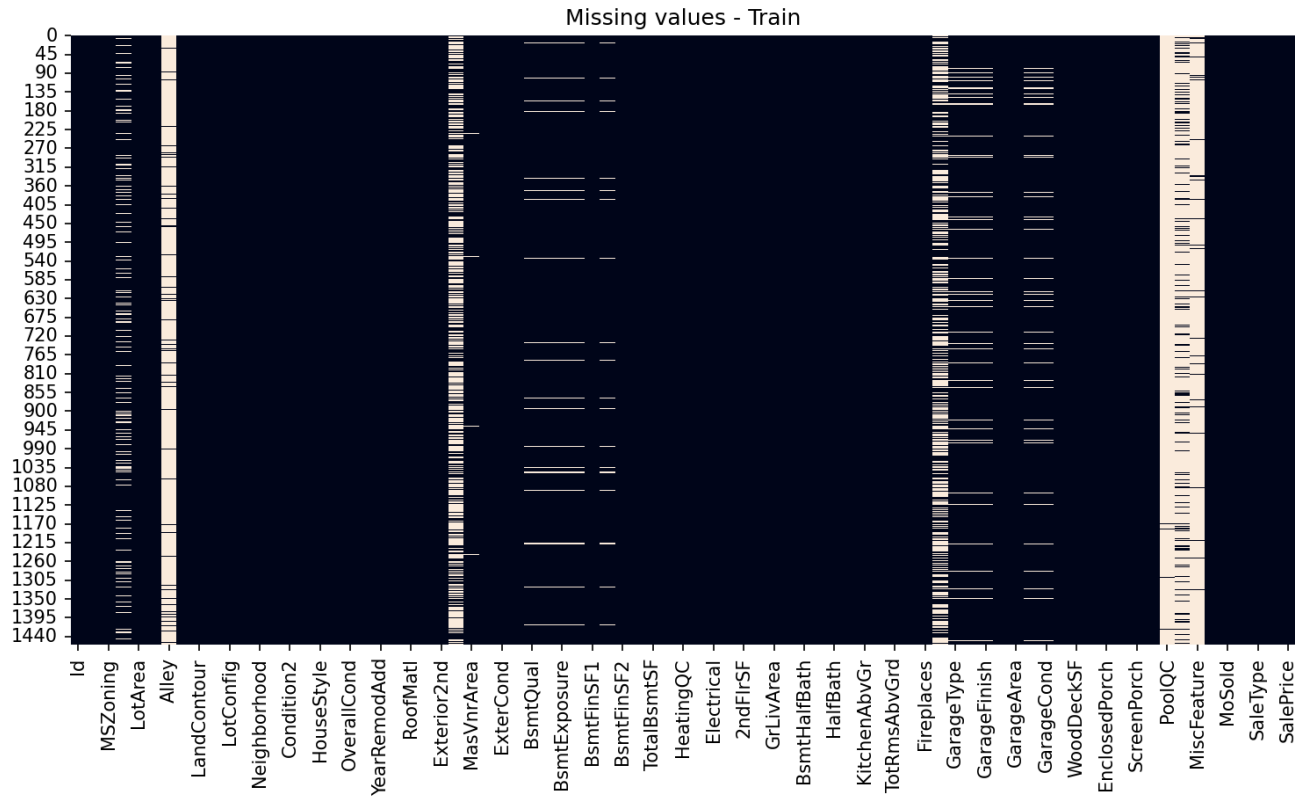
Qua kiểm tra thống kê trên tập huấn luyện , nhóm ghi nhận có tổng cộng 19 đặc trưng xuất hiện dữ liệu khuyết thiếu ở các mức độ khác nhau:

- **Nhóm đặc trưng khuyết thiếu nghiêm trọng (trên 70%):** Chiếm tỷ lệ cao nhất là PoolQC (1453 mẫu thiếu), tiếp đến là MiscFeature (1406 mẫu thiếu), Alley (1369 mẫu thiếu) và Fence (1179 mẫu thiếu).
- **Nhóm đặc trưng khuyết thiếu trung bình:** Biến FireplaceQu (690 mẫu thiếu) và LotFrontage (259 mẫu thiếu).
- Các cột nhà để xe, tầng hầm, MasVnrArea và Electrical có số lượng thiếu ở các mức khác nhau. Riêng MasVnrType thiếu nhiều hơn đáng kể nên không thể xếp vào nhóm dưới 6%. Bảng kiểm tra số lượng thiếu cần được dùng để phân loại chính xác trước khi điền giá trị.

Các cột có missing values trong train:

PoolQC	1453
MiscFeature	1406
Alley	1369
Fence	1179
MasVnrType	872
FireplaceQu	690
LotFrontage	259
GarageType	81
GarageYrBlt	81
GarageFinish	81
GarageQual	81
GarageCond	81
BsmtExposure	38
BsmtFinType2	38
BsmtQual	37
BsmtCond	37
BsmtFinType1	37
MasVnrArea	8
Electrical	1
dtype:	int64

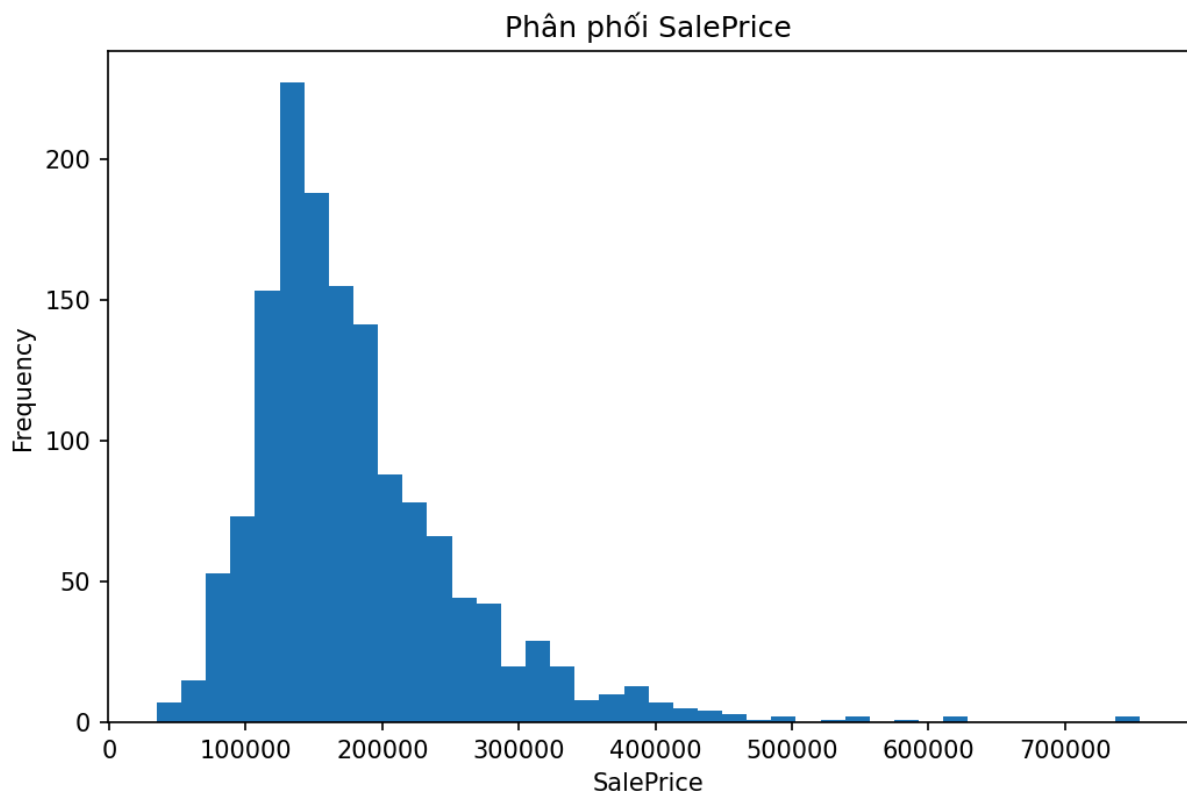
Để có góc nhìn trực quan toàn diện về sự phân bố của các giá trị khuyết thiếu trên cả hai tập dữ liệu, nhóm đã dựng biểu đồ nhiệt (Heatmap) thông qua thư viện Seaborn.



Biểu đồ nhiệt cho thấy một số cột khuyết nhiều ở cả train và test, nhất là *PoolQC*, *MiscFeature*, *Alley* và *Fence*. Riêng với nhiều cột về tiện ích, giá trị NA có thể mang ý nghĩa căn nhà không có tiện ích đó; tuy nhiên chỉ từ biểu đồ không thể kết luận cơ chế thiếu dữ liệu là MCAR hay MNAR. Notebook hiện tại dùng cách điền thiếu theo kiểu dữ liệu để tạo ma trận đặc trưng nhất quán.

2.1. Khảo sát phân bố của biến mục tiêu (SalePrice)

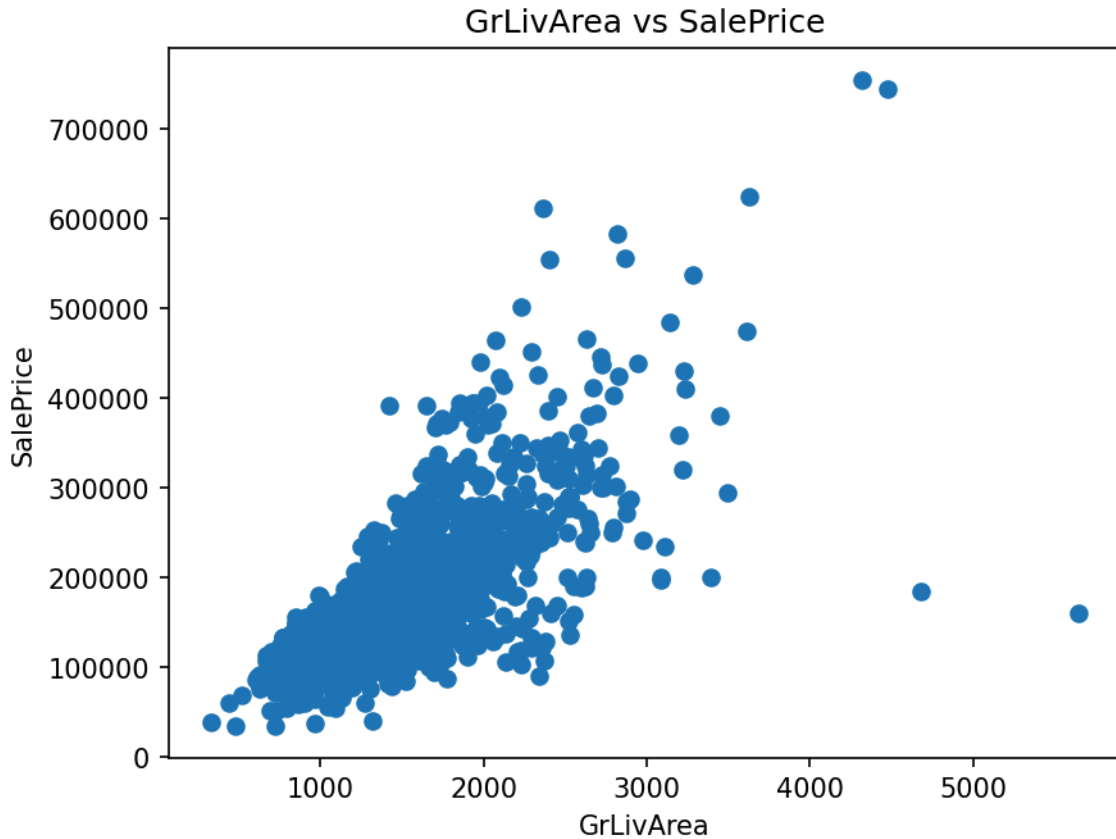
Do đây là bài toán hồi quy định giá, phân bố xác suất của biến mục tiêu đóng vai trò quyết định đến độ ổn định của quá trình tối ưu hóa trọng số mạng nơ-ron. Khi trực quan hóa biến SalePrice bằng Histogram, hình dạng phân phối thể hiện rõ tính chất bất đối xứng:



Đánh giá toán học: Biểu đồ cho thấy phân bố giá nhà bị lệch phải với phần đuôi kéo dài về phía các mức giá cao. Kết quả tính toán hệ số lệch đạt skewness = 1.8829. Phần lớn các giao dịch tập trung trong khoảng từ 130.000 USD đến 210.000 USD, trong khi chỉ có một số ít căn biệt thự cao cấp có giá vượt trên 500.000 USD. Sự chênh lệch này là căn cứ thực nghiệm quan trọng để nhóm quyết định thực hiện phép biến đổi logarit cho biến mục tiêu ở bước tiền xử lý nhằm ổn định phương sai cho mô hình.

2.2. Khảo sát tương quan và phát hiện ngoại lệ (Outliers)

Theo phân tích kinh tế lượng, diện tích sinh hoạt trên mặt đất (*GrLivArea*) là một trong những đặc trưng có mối tương quan tuyến tính mạnh nhất đối với giá bán nhà. Nhóm sử dụng biểu đồ phân tán (Scatter Plot) để quan sát đồng thời hai biến này:



Hiện tượng ghi nhận: Đa số các điểm dữ liệu phân bố tập trung theo một đường xu hướng dốc lên khá rõ ràng. Tuy nhiên, ở góc dưới bên phải đồ thị xuất hiện các điểm dị biệt cực đoạn: những căn nhà có diện tích sinh hoạt rất lớn (vượt 4.000 feet) nhưng giá bán lại thấp một cách bất thường (dưới 200.000 USD).

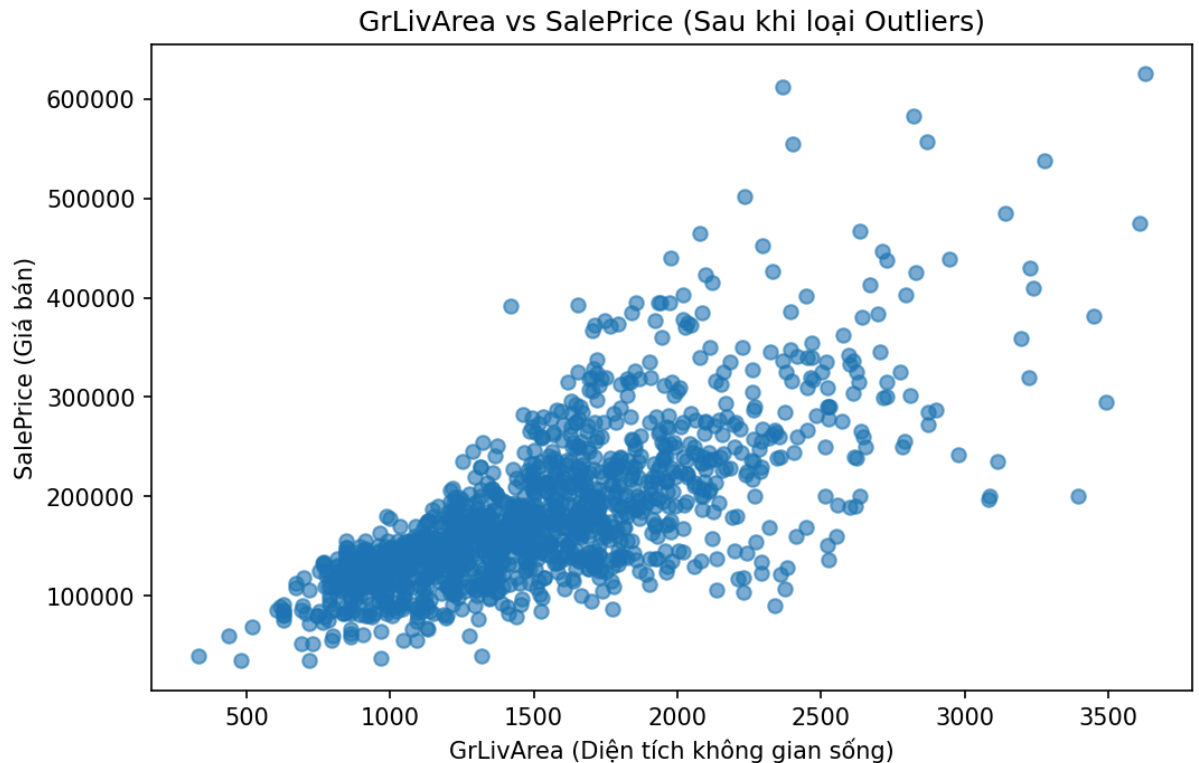
3. TIỀN XỬ LÝ DỮ LIỆU (DATA PREPARATION)

Dựa trên các kết quả khám phá ở giai đoạn trước, nhóm tiến hành các bước làm sạch và chuẩn bị dữ liệu theo một pipeline chặt chẽ trước khi đưa vào PyTorch.

3.1. Loại bỏ các điểm ngoại lệ (Outlier Removal)

- **Quy tắc thực hiện:** nhóm loại bỏ dòng train có GrLivArea trên 4.000 sq ft để giảm ảnh hưởng của các giao dịch diện tích đặc biệt lớn đến việc fit mô hình. Đây là một lựa chọn xử lý ngoại lệ theo dữ liệu; báo cáo không suy đoán nguyên nhân bán giá thấp của từng căn nhà nếu không có thông tin giao dịch.
- **Thực thi:** Lọc bỏ các dòng thỏa mãn điều kiện $\text{GrLivArea} > 4000$ trên tập train. Kết quả: loại bỏ chính xác 4 điểm dị biệt, kích thước tập train giảm từ 1460 dòng xuống còn 1456 dòng.

- **Nguyên tắc bảo toàn tập test:** Tuyệt đối không can thiệp xóa dòng trên tập test để đảm bảo định dạng và số lượng dòng dự đoán theo đúng yêu cầu chấm điểm tự động của cuộc thi Kaggle.



Biểu đồ sau khi xử lý cho thấy vùng dữ liệu diện tích lớn đã trở nên đồng nhất, tương quan tuyến tính giữa diện tích sống và giá bán được thể hiện rõ nét hơn.

3.2. Đóng gói hàm tiền xử lý dữ liệu (`preprocess_housing`)

Nhóm xây dựng hàm `preprocess_housing` để tạo cùng hệ đặc trưng cho train và test. Notebook ghép hai tập sau khi bỏ nhãn `SalePrice` của train, xử lý thiếu và mã hóa, rồi tách lại theo số dòng ban đầu. Việc ghép trước khi chia validation giúp số cột khớp nhau nhưng cũng là giới hạn phương pháp: thống kê điền khuyết có thể đã sử dụng đặc trưng của tập validation.

- **Gộp train và test:** nhóm bỏ `SalePrice` khỏi train, nối tạm với test để các giá trị phân loại được mã hóa thành cùng danh sách cột. Sau đó notebook tách lại theo đúng số dòng của train. Bước này chỉ dùng đặc trưng đầu vào, không dùng giá bán của test.
- **Lựa chọn và xử lý thuộc tính:**
 - *Loại bỏ cột:* notebook bỏ `Id` khỏi đầu vào mô hình và đồng thời bỏ bốn cột thiếu trên 70% là `Alley`, `PoolQC`, `Fence`, `MiscFeature`.

Cột *Id* của *test* vẫn được giữ ở *DataFrame* gốc để ghép vào file nộp.

- Các cột *Alley*, *PoolQC*, *Fence* và *MiscFeature* không được giữ trong ma trận đặc trưng của notebook hiện tại vì tỷ lệ thiếu cao. Đây là một quyết định đơn giản hóa tiền xử lý; nhóm có thể thử cách mã hóa trạng thái không có tiện ích trong nghiên cứu tiếp theo.
- Phân loại kiểu dữ liệu: notebook nhận diện cột số theo kiểu *int64* hoặc *float64*, và cột phân loại theo kiểu *object*. Nó không chuyển riêng *MSSubClass* hoặc *MoSold* thành chuỗi trong lần chạy hiện tại; hai cột này được xử lý như biến số.

- **Chiến lược điền dữ liệu thiếu trong notebook hiện tại:**

- Cột phân loại còn thiếu được điền bằng giá trị xuất hiện nhiều nhất (*mode*) của từng cột trong dữ liệu đã ghép. Cách này bảo đảm đầu vào không còn *NaN*, nhưng chưa bảo toàn ý nghĩa *None* riêng cho các trường hợp không có tiện ích.
- Các cột số còn thiếu được điền bằng giá trị trung bình (*mean*) của từng cột trong dữ liệu đã ghép. Notebook hiện tại không gán 0 riêng cho diện tích hay số lượng tiện ích bị thiếu.
- *LotFrontage* được điền bằng trung bình của toàn cột trong hàm tiền xử lý hiện tại, không phải trung vị theo *Neighborhood*. Đây là một bước có thể cải thiện về sau nếu nhóm thử quy trình điền thiếu theo khu vực.
- Tóm lại, các cột số thiếu được điền bằng *mean* và các cột phân loại thiếu được điền bằng *mode*. Sau bước này, notebook kiểm tra tổng số ô còn thiếu bằng 0 rồi áp dụng *one-hot encoding* với *drop_first=True*.
- Kết quả: Kiểm tra *isnull().sum().sum()* sau khi điền đạt bằng 0 (triệt tiêu hoàn toàn giá trị *NaN*, bảo vệ thuật toán *Gradient Descent* không bị lỗi gây đạo hàm).

- **Mã hóa đặc trưng phân loại (One-Hot Encoding):**

Sử dụng hàm `pd.get_dummies(all_data, drop_first=True, dtype=float)` để chuyển toàn bộ các chuỗi văn bản thành dạng số thực 1.0 và 0.0. Tham số `drop_first=True` giúp tránh hiện tượng cộng tuyến đa biến (*multicollinearity*), đồng thời tham số `dtype=float` giúp tương thích hoàn toàn với thư viện *PyTorch*.

1.

2. **Biến đổi logarit cho biến mục tiêu:**

Áp dụng công thức :

$$y = \ln(1 + \text{SalePrice})$$

Thông qua hàm (`np.log1p()`) cho nhãn huấn luyện.

Phép biến đổi `log1p` giúp giảm độ lệch phải của biến `SalePrice` và làm phân phối target ổn định hơn trong quá trình huấn luyện. Việc sử dụng target trên thang `log1p` cũng phù hợp với cách Kaggle đánh giá bằng RMSE trên logarit của giá bán.

```
def preprocess_housing(train_df, test_df):
    n_train = len(train_df)

    # 1. Tách nhãn và gộp tập train (bỏ SalePrice) và test để đồng bộ không gian đặc trưng
    train_features = train_df.drop("SalePrice", axis=1)
    all_data = pd.concat([train_features, test_df], axis=0).reset_index(drop=True)

    # Bỏ cột Id vì không mang giá trị quy luật
    if 'Id' in all_data.columns:
        all_data = all_data.drop(columns=['Id'])

    # Chuyển các cột số mang bản chất phân loại thời vụ/kiến trúc sang chuỗi
    for cat_num_col in ['MSSubClass', 'MoSold']:
        if cat_num_col in all_data.columns:
            all_data[cat_num_col] = all_data[cat_num_col].astype(str)

    # 2. Xử lý các cột phân loại mà NA mang nghĩa "Không có tiện ích" -> Điền chuỗi 'None'
    none_cols = [
        'PoolQC', 'MiscFeature', 'Alley', 'Fence', 'FireplaceQu',
        'GarageType', 'GarageFinish', 'GarageQual', 'GarageCond',
        'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2',
        'MasVnrType'
    ]
    for col in none_cols:
        if col in all_data.columns:
            all_data[col] = all_data[col].fillna('None')

    # 3. Xử lý các cột số liên quan đến tầng hầm & nhà xe (Không có tiện ích -> Điền 0)
    zero_cols = [
        'GarageYrBlt', 'GarageArea', 'GarageCars',
        'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF',
        'BsmtFullBath', 'BsmtHalfBath', 'MasVnrArea'
    ]
    for col in zero_cols:
        if col in all_data.columns:
            all_data[col] = all_data[col].fillna(0)

    # 4. Điền LotFrontage theo trung vị của từng khu vực (Neighborhood)
    if 'LotFrontage' in all_data.columns:
        all_data['LotFrontage'] = all_data.groupby('Neighborhood')['LotFrontage'].transform(
            lambda x: x.fillna(x.median())
        )
        all_data['LotFrontage'] = all_data['LotFrontage'].fillna(all_data['LotFrontage'].median())

    # 5. Điền thiếu ngẫu nhiên cho các cột phân loại còn lại (MSZoning, Utilities, KitchenQual, Electrical...)
    cat_remaining = all_data.select_dtypes(include=['object', 'category', 'string']).columns
    for col in cat_remaining:
        if all_data[col].isnull().sum() > 0:
            all_data[col] = all_data[col].fillna(all_data[col].mode()[0])
```

```

# 6. Điền thiếu ngẫu nhiên cho các cột số còn lại nếu có
num_remaining = all_data.select_dtypes(include=['int64', 'float64']).columns
for col in num_remaining:
    if all_data[col].isnull().sum() > 0:
        all_data[col] = all_data[col].fillna(all_data[col].median())

# 7. One-Hot Encoding (ép dtype=float để tương thích tuyệt đối với StandardScaler và PyTorch)
all_data = pd.get_dummies(all_data, drop_first=True, dtype=float)

# 8. Tách lại train và test
X = all_data.iloc[:n_train, :].copy()
X_test_kaggle = all_data.iloc[n_train:, :].copy()

# Biến đổi log1p cho biến mục tiêu SalePrice
y = np.log1p(train_df["SalePrice"]).copy()

return X, y, X_test_kaggle

# Chạy tiền xử lý
X, y, X_test_kaggle = preprocess_housing(train, test)

print("=" * 60)
print("KẾT QUẢ TIỀN XỬ LÝ")
print("=" * 60)
print(f"Shape X (Train)           : {X.shape}")
print(f"Shape y (Target Train)      : {y.shape}")
print(f"Shape X_test_kaggle (Test): {X_test_kaggle.shape}")
print(f"Đồng bộ số lượng cột       : {X.shape[1] == X_test_kaggle.shape[1]}")
print(f"Missing values còn lại     : {X.isnull().sum().sum()}")
print("=" * 60)

```

Sau khi tách ngược all_data về hai phần ban đầu, nhóm thu được:

- Tập đặc trưng huấn luyện X: 1.456 mẫu và 234 cột sau khi loại cột, điền khuyết và one-hot encoding.
- Biến mục tiêu huấn luyện y: Gồm 1456 giá trị nhãn đã chuyển đổi log.
- Tập đặc trưng kiểm thử X_test_kaggle: 1.459 mẫu và 234 cột, khớp đúng thứ tự cột với X.

```

=====
KẾT QUẢ TIỀN XỬ LÝ
=====
Shape X (Train)           : (1456, 282)
Shape y (Target Train)      : (1456,)
Shape X_test_kaggle (Test): (1459, 282)
Đồng bộ số lượng cột       : True
Missing values còn lại     : 0
=====

```

3.3. Phân chia tập dữ liệu, Chuẩn hóa đặc trưng & Chuyển đổi PyTorch Tensors

Chuẩn bị dữ liệu đầu vào cho kiến trúc mạng MLP :

- **Phân chia Train/Validation:** Tách ngẫu nhiên tập dữ liệu thành phần huấn luyện nội bộ (X_train, y_train) chiếm 70% (1019 mẫu) và phần kiểm định (X_val, y_val) chiếm 30% (437 mẫu). Do bài toán dự đoán giá nhà là bài toán hồi quy liên tục, nhóm **không sử dụng tham số stratify=y** .
- **Chuẩn hóa tỷ lệ bằng StandardScaler:** Các đặc trưng đầu vào có đơn vị đo rất khác nhau . Nhóm áp dụng StandardScaler để đưa toàn bộ về phân phối có kỳ vọng và phương sai $\sigma^2 = 1$. Để chống hiện tượng rò rỉ dữ liệu , scaler chỉ được học tham số trên X_train (fit_transform), sau đó áp dụng phép chiếu transform cho X_val và X_test_kaggle.
- **Tensor hóa và đóng gói DataLoader:** Toàn bộ dữ liệu được chuyển đổi thành torch.Tensor với kiểu dữ liệu torch.float32. Nhân mục tiêu y được chuyển thành tensor cột hai chiều (N, 1) bằng lệnh .view(-1, 1) để tương thích hoàn hảo với đầu ra của nơ-ron hồi quy trong PyTorch, tránh lỗi Broadcasting trong hàm tính loss nn.MSELoss(). Tập train được nạp vào DataLoader với kích thước lô batch_size = 32 và bật cờ xáo trộn ngẫu nhiên (shuffle = True).
- **Kích thước đầu vào của MLP là input_size = X_train_t.shape[1] = 234, tương ứng với 234 đặc trưng sau tiền xử lý.**

1. Phân chia tập Train và Validation

```
X_train, X_val, y_train, y_val = train_test_split(
    X,
    y,
    test_size=params["data"]["validation_size"],
    random_state=params["data"]["random_state"]
)
```

2. Chuẩn hóa đặc trưng (StandardScaler)

```
scaler = StandardScaler()
```

Chỉ fit trên X_train để tránh rò rỉ dữ liệu

```
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_kaggle_scaled = scaler.transform(X_test_kaggle)
```

```
print("Kích thước sau khi chia tập:")
print(f"- X_train: {X_train.shape}, y_train: {y_train.shape}")
print(f"- X_val : {X_val.shape}, y_val : {y_val.shape}")
```

```

# 3. Chuyển đổi sang PyTorch Tensor (float32)
X_train_t = torch.tensor(X_train_scaled, dtype=torch.float32)
y_train_t = torch.tensor(y_train.values, dtype=torch.float32).view(-1, 1)

X_val_t = torch.tensor(X_val_scaled, dtype=torch.float32)
y_val_t = torch.tensor(y_val.values, dtype=torch.float32).view(-1, 1)

X_kaggle_t = torch.tensor(X_test_kaggle_scaled, dtype=torch.float32)

# 4. Tạo DataLoader cho quá trình huấn luyện
train_loader = DataLoader(
    TensorDataset(X_train_t, y_train_t),
    batch_size=params["train"]["batch_size"],
    shuffle=True
)

# 5. Lưu và in thông tin bàn giao cho Member 2 & Member 3
input_size = X_train_t.shape[1]

print("=" * 70)

print("=" * 70)
print(f"- Kích thước Tensor X_train_t : {X_train_t.shape}")
print(f"- Kích thước Tensor y_train_t : {y_train_t.shape}")
print(f"- Kích thước Tensor X_val_t : {X_val_t.shape}")
print(f"- Kích thước Tensor y_val_t : {y_val_t.shape}")
print(f"- Kích thước Tensor X_kaggle_t : {X_kaggle_t.shape}")
print(f"- Số lượng batch trong train_loader : {len(train_loader)}")
print("-" * 70)
print(f">>> THÔNG SỐ INPUT_SIZE CHO MODEL PYTORCH: {input_size} <<<")
print("-" * 70)
print(f"Danh sách {len(X.columns)} đặc trưng đầu vào (Features):")
print(X.columns.tolist())
print("=" * 70)

```

Kích thước sau khi chia tập:

- X_train: (1019, 282), y_train: (1019,)
- X_val : (437, 282), y_val : (437,)

```

=====
- Kích thước Tensor X_train_t : torch.Size([1019, 234])
- Kích thước Tensor y_train_t : torch.Size([1019, 1])
- Kích thước Tensor X_val_t : torch.Size([437, 234])
- Kích thước Tensor y_val_t : torch.Size([437, 1])
- Kích thước Tensor X_kaggle_t : torch.Size([1459, 234])
- Số lượng batch trong train_loader : 32
=====
>>> THÔNG SỐ INPUT_SIZE CHO MODEL PYTORCH: 234 <<<
=====
Danh sách 234 đặc trưng đầu vào (Features):
['MSSubClass', 'LotFrontage', 'LotArea', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd', 'MasVnrArea', 'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'TotRmsAbvGrd', 'Fireplaces', 'GarageYrBlt', 'GarageCars', 'GarageArea', 'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'MiscVal', 'MoSold', 'YrSold', 'MSZoning_FV', 'MSZoning_RH', 'MSZoning_RL', 'MSZoning_RM', 'Street_Pave', 'LotShape_IR2', 'LotShape_IR3', 'LotShape_Reg', 'LandContour_HLS', 'LandContour_Low', 'LandContour_Lvl', 'Utilities_NoSeWa', 'LotConfig_CulDSac', 'LotConfig_FR2', 'LotConfig_FR3', 'LotConfig_Inside', 'LandSlope_Mod', 'LandSlope_Sev', 'Neighborhood_Blueste', 'Neighborhood_BrDale', 'Neighborhood_BrkSide', 'Neighborhood_ClearCr', 'Neighborhood_CollgCr', 'Neighborhood_Crawfor', 'Neighborhood_Edwards', 'Neighborhood_Gilbert', 'Neighborhood_IDOTRR', 'Neighborhood_MeadowV', 'Neighborhood_MitcheL', 'Neighborhood_NAmes', 'Neighborhood_NPKVill', 'Neighborhood_NMAmes', 'Neighborhood_NoRidge', 'Neighborhood_NridgHt', 'Neighborhood_OldTown', 'Neighborhood_SWISU', 'Neighborhood_Sawyer', 'Neighborhood_SawyerW', 'Neighborhood_Somerst', 'Neighborhood_StoneBr', 'Neighborhood_Timber', 'Neighborhood_Veenker', 'Condition1_Feeder', 'Condition1_Norm', 'Condition1_PosA', 'Condition1_PosH', 'Condition1_RRAe', 'Condition1_RRAh', 'Condition1_RRNe', 'Condition1_RRNh', 'Condition1_Feeder', 'Condition2_Norm', 'Condition2_PosA', 'Condition2_PosH', 'Condition2_RRAe', 'Condition2_RRAh', 'Condition2_RRNh', 'BldgType_2fmCon', 'BldgType_Duplex', 'BldgType_Twnhs', 'BldgType_TwnhsE', 'HouseStyle_1.5Unf', 'HouseStyle_1Story', 'HouseStyle_2.5Fin', 'HouseStyle_2.5Unf', 'HouseStyle_2Story', 'HouseStyle_SFoyer', 'HouseStyle_SLVl', 'RoofStyle_Gable', 'RoofStyle_Gambrel', 'RoofStyle_Hip', 'RoofStyle_Mansard', 'RoofStyle_Shed', 'RoofMatl_Membran', 'RoofMatl_Metal', 'RoofMatl_Roll', 'RoofMatl_Tar&Grv', 'RoofMatl_WdShake', 'RoofMatl_WdShngl', 'Exterior1st_AsphShn', 'Exterior1st_BrkCmn', 'Exterior1st_BrkFace', 'Exterior1st_CBlock', 'Exterior1st_CBlock', 'Exterior1st_CemntBd', 'Exterior1st_HdBoard', 'Exterior1st_ImStucc', 'Exterior1st_MetalSd', 'Exterior1st_Plywood', 'Exterior1st_Stone', 'Exterior1st_Stucco', 'Exterior1st_VinylSd', 'Exterior1st_Wd Sdng', 'Exterior1st_WdShngl', 'Exterior2nd_AsphShn', 'Exterior2nd_Brk Cmn', 'Exterior2nd_BrkFace', 'Exterior2nd_CBlock', 'Exterior2nd_CmentBd', 'Exterior2nd_HdBoard', 'Exterior2nd_ImStucc', 'Exterior2nd_MetalSd', 'Exterior2nd_Other', 'Exterior2nd_Plywood', 'Exterior2nd_Stone', 'Exterior2nd_Stucco', 'Exterior2nd_VinylSd', 'Exterior2nd_Wd Sdng', 'Exterior2nd_Wd Shngl', 'MasVnrType_BrkFace', 'MasVnrType_Stone', 'ExterQual_Fa', 'ExterQual_Gd', 'ExterQual_TA', 'ExterCond_Fa', 'ExterCond_Gd', 'ExterCond_Po', 'ExterCond_TA', 'Foundation_CBlock', 'Foundation_PConc', 'Foundation_Slab', 'Foundation_Stone', 'Foundation_Wood', 'BsmtQual_Fa', 'BsmtQual_Gd', 'BsmtQual_TA', 'BsmtCond_Gd', 'BsmtCond_Po', 'BsmtCond_TA', 'BsmtExposure_Gd', 'BsmtExposure_Mn', 'BsmtExposure_No', 'BsmtFinType1_BLQ', 'BsmtFinType1_GLQ', 'BsmtFinType1_LwQ', 'BsmtFinType1_Rec', 'BsmtFinType1_Unf', 'BsmtFinType2_BLQ', 'BsmtFinType2_GLQ', 'BsmtFinType2_LwQ', 'BsmtFinType2_Rec', 'BsmtFinType2_Unf', 'Heating_GasA', 'Heating_GasW', 'Heating_Grav', 'Heating_OTHW', 'Heating_Wall', 'HeatingQC_Fa', 'HeatingQC_Gd', 'HeatingQC_Po', 'HeatingQC_TA', 'CentralAir_Y', 'Electrical_FuseF', 'Electrical_FuseP', 'Electrical_Mix', 'Electrical_SBrkr', 'KitchenQual_Fa', 'KitchenQual_Gd', 'KitchenQual_TA', 'Functional_Maj2', 'Functional_Min1', 'Functional_Min2', 'Functional_Mod', 'Functional_Sev', 'Functional_Typ', 'FireplaceQu_Fa', 'FireplaceQu_Gd', 'FireplaceQu_Po', 'FireplaceQu_TA', 'GarageType_Attchd', 'GarageType_Basment', 'GarageType_BuiltIn', 'GarageType_CarPort', 'GarageType_Detchd', 'GarageFinish_RFn', 'GarageFinish_Unf', 'GarageQual_Fa', 'GarageQual_Gd', 'GarageQual_Po', 'GarageQual_TA', 'GarageCond_Fa', 'GarageCond_Gd', 'GarageCond_Po', 'GarageCond_TA', 'PavedDrive_P', 'PavedDrive_Y', 'SaleType_CWD', 'SaleType_Con', 'SaleType_ConLD', 'SaleType_ConLI', 'SaleType_ConLw', 'SaleType_New', 'SaleType_Oth', 'SaleType_PeWD', 'SaleCondition_AdjLand', 'SaleCondition_Alloca', 'SaleCondition_Family', 'SaleCondition_Normal', 'SaleCondition_Partial']
=====

```

Do bước imputation và one-hot encoding được thực hiện trên all_data trước khi tách validation, kết quả validation cần được diễn giải thận trọng; StandardScaler được fit riêng trên X_train nhưng thống kê imputation không được giới hạn trong X_train.

4. XÂY DỰNG MÔ HÌNH (MODELING)

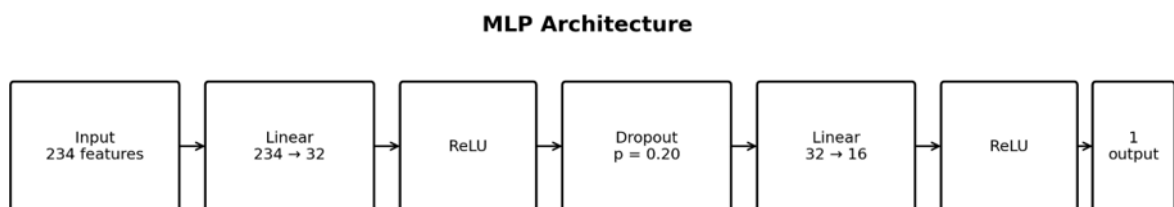
Sau khi hoàn thành quá trình khám phá và tiền xử lý dữ liệu, nhóm tiến hành xây dựng các mô hình dự đoán giá nhà. Mục tiêu của giai đoạn Modeling là xây dựng một mô hình học sâu bằng PyTorch theo yêu cầu của bài tập, đồng thời thực hiện các thí nghiệm bổ sung với những mô hình học máy khác nhằm tìm kiếm cấu hình có khả năng dự đoán tốt hơn.

Quá trình xây dựng mô hình được thực hiện theo hướng tăng dần mức độ phức tạp. Trước tiên, nhóm xây dựng một mô hình Multilayer Perceptron (MLP) bằng PyTorch làm mô hình cơ sở. Sau đó, các siêu tham số của MLP được điều chỉnh thông qua quá trình tuning. Tiếp theo, nhóm thử nghiệm thêm XGBoost, CatBoost và Lasso nhằm khai thác các cơ chế học khác nhau. Cuối cùng, dự đoán của các mô hình được kết hợp bằng phương pháp weighted ensemble. Ngoài ra, nhóm thực hiện 5-Fold Out-of-Fold (OOF) để kiểm tra độ ổn định của các mô hình và trọng số ensemble.

4.1. Xây dựng mô hình MLP bằng PyTorch

4.1.1 Kiến trúc mô hình ban đầu

MLP được sử dụng làm mô hình học sâu cơ sở của bài toán. Kiến trúc được cài đặt bằng PyTorch và gồm một lớp đầu vào, hai lớp ẩn và một lớp đầu ra. Ở phiên bản baseline, số nơ-ron của hai lớp ẩn lần lượt là 32 và 16. Hàm kích hoạt ReLU được sử dụng sau mỗi lớp ẩn, trong khi Dropout với xác suất 0,20 được đặt sau lớp ẩn thứ nhất để hạn chế overfitting. Lớp đầu ra chỉ có một nơ-ron, tương ứng với giá trị dự đoán của $\log(1 + \text{SalePrice})$.



Hình 4.1. Kiến trúc mô hình MLP baseline trong PyTorch

Biến mục tiêu được sử dụng trong quá trình huấn luyện là:

$$y = \log(1 + \text{SalePrice})$$

Do đó, đầu ra của MLP cũng nằm trên thang đo $\log(1 + \text{SalePrice})$. Khi cần chuyển kết quả dự đoán trở về đơn vị giá nhà thực tế, nhóm sử dụng phép biến đổi ngược:

$$\text{SalePrice} = \exp(\hat{y}) - 1$$

Việc huấn luyện trên thang logarit giúp giảm ảnh hưởng của sự lệch phải trong phân phối giá nhà và đồng thời phù hợp với tiêu chí đánh giá RMSE trên thang logarit của cuộc thi.

4.1.2 Kiến trúc mô hình sau Tuning

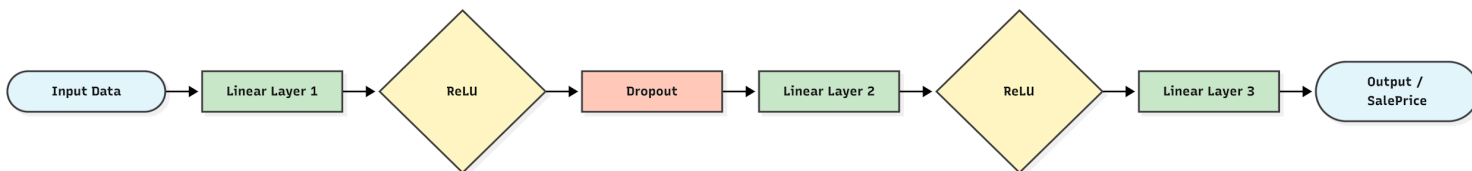
Kiến trúc MLP được xây dựng bằng thư viện PyTorch. Mô hình gồm một lớp đầu vào, hai lớp ẩn và một lớp đầu ra.

Kiến trúc tổng quát:

Input → Linear → ReLU → Dropout → Linear → ReLU → Linear → Output

Trong cấu hình tốt nhất sau quá trình tuning, hai lớp ẩn lần lượt có 24 và 12 nơ-ron. Dropout được đặt sau lớp ẩn thứ nhất với xác suất 0,20.

Sơ đồ kiến trúc có thể trình bày như sau:



Hình 4.2. Kiến trúc mô hình MLP sau tuning

Nguồn: Nhóm thực hiện

Lớp Linear thực hiện phép biến đổi tuyến tính giữa các đặc trưng. Hàm kích hoạt ReLU được sử dụng để đưa tính phi tuyến vào mạng, cho phép mô hình biểu diễn các quan hệ phức tạp giữa các đặc trưng và giá nhà. Lớp Dropout được sử dụng như một kỹ thuật regularization nhằm hạn chế hiện tượng quá khớp.

4.1.3 Hàm mất mát

Do bài toán là hồi quy và mục tiêu được biểu diễn trên thang logarit, nhóm sử dụng Mean Squared Error (MSE) làm hàm mất mát trong quá trình huấn luyện:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Trong đó:

- N là số lượng mẫu;
- y_i là giá trị thực tế trên thang log1p;
- $\hat{y}_i$ là giá trị dự đoán của mô hình.

MSE phù hợp với quá trình tối ưu vì RMSE được tính bằng căn bậc hai của MSE:

$$RMSE = \sqrt{MSE}$$

Do đó, giảm MSE trong quá trình huấn luyện đồng thời hướng mô hình đến việc giảm RMSE trên tập validation.

4.1.4. Bộ tối ưu và cơ chế regularization

Nhóm sử dụng thuật toán Adam để cập nhật trọng số của mạng. Adam được lựa chọn vì có khả năng điều chỉnh tốc độ học của từng tham số dựa trên các moment của gradient, giúp quá trình tối ưu ổn định hơn so với việc sử dụng learning rate cố định đơn giản.

Bên cạnh Dropout, nhóm sử dụng weight_decay trong optimizer Adam như một cơ chế regularization. Hai cơ chế này được sử dụng nhằm giảm nguy cơ mô hình học quá sát dữ liệu huấn luyện.

4.2. Huấn luyện mô hình

4.2.1. Quy trình huấn luyện

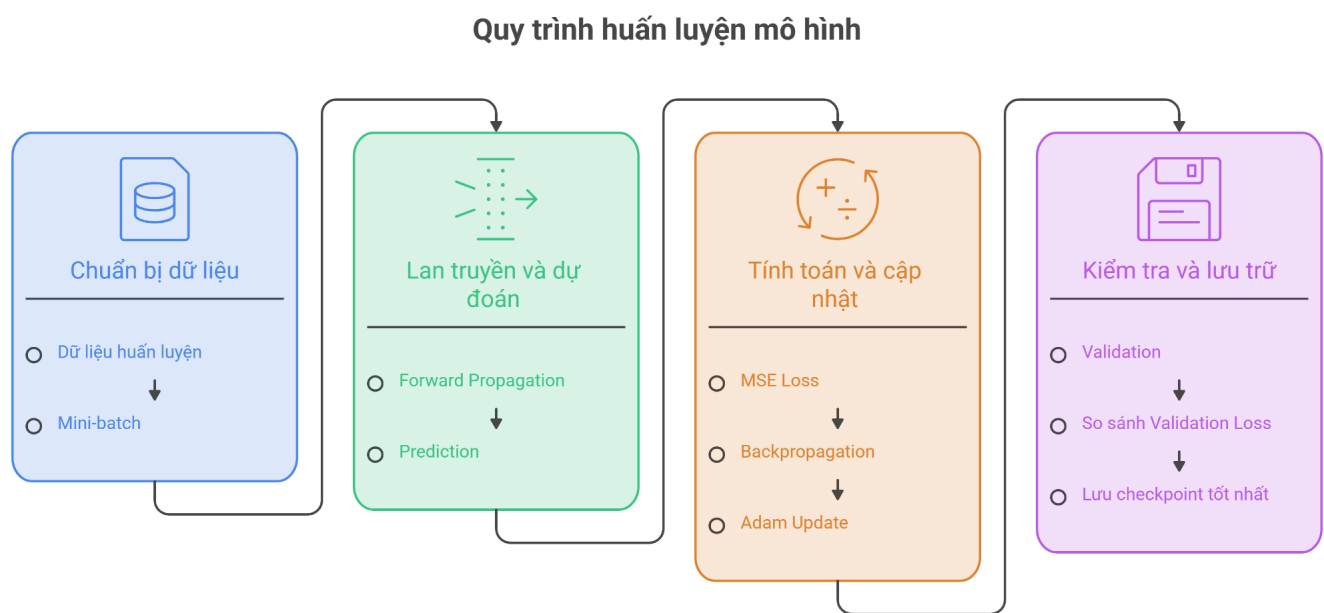
Dữ liệu huấn luyện được đưa vào DataLoader theo mini-batch với batch size bằng 32. Trong mỗi bước cập nhật, mô hình thực hiện forward propagation để tạo dự đoán, tính MSE loss, lan truyền ngược gradient và cập nhật trọng số bằng Adam. Sau mỗi epoch, mô hình được chuyển sang chế độ đánh giá để tính validation loss. Khi validation loss đạt giá trị thấp nhất, trạng thái của mô hình được lưu lại thành checkpoint tốt nhất.

Trong mỗi bước cập nhật, mô hình thực hiện lần lượt các thao tác:

1. Nhận một mini-batch dữ liệu đầu vào.
2. Thực hiện forward propagation để tạo dự đoán.
3. Tính MSE loss giữa dự đoán và giá trị thực tế.
4. Thực hiện backpropagation để tính gradient.
5. Cập nhật trọng số bằng thuật toán Adam.

Quy trình này được lặp lại trong nhiều epoch cho đến khi hoàn thành số vòng lặp được thiết lập.

Có thể minh họa quá trình huấn luyện bằng sơ đồ:



Hình 4.3. Quy trình huấn luyện và lựa chọn checkpoint tốt nhất

4.2.2. Theo dõi validation

Sau mỗi epoch, nhóm chuyển mô hình sang chế độ đánh giá và tính validation loss trên tập validation. Giá trị validation loss được sử dụng để theo dõi khả năng tổng quát hóa của mô hình.

Thay vì chỉ sử dụng trọng số ở epoch cuối cùng, nhóm lưu lại trạng thái của mô hình tại thời điểm validation loss đạt giá trị thấp nhất. Cách này giúp hạn chế việc sử dụng một mô hình đã bắt đầu overfitting ở những epoch cuối.

Checkpoint của mô hình MLP baseline được lưu dưới dạng `best_model.pt`; sau quá trình tuning, mô hình tốt nhất được lưu riêng dưới dạng `best_tuned_model.pt`.

Trong các thí nghiệm tiếp theo, cấu hình và checkpoint tốt nhất được quản lý riêng để có thể tái sử dụng mà không cần huấn luyện lại từ đầu.

4.2.3. Kết quả MLP baseline

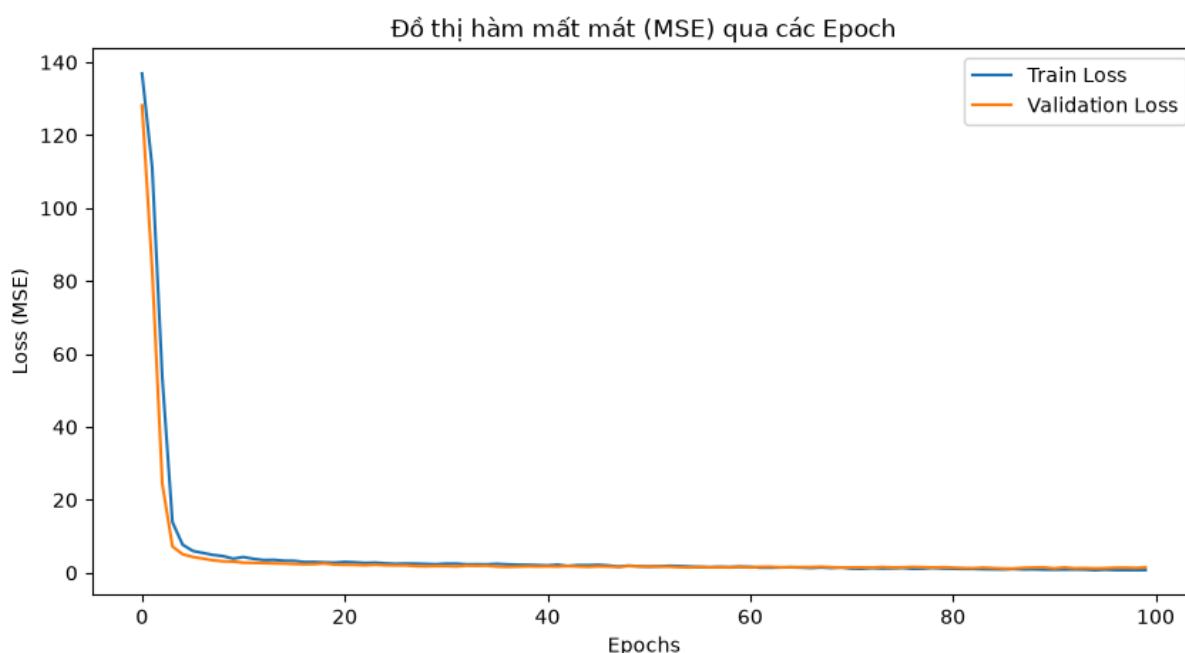
Sau khi huấn luyện mô hình MLP baseline với cấu hình ban đầu gồm 32 nơ-ron ở lớp ẩn thứ nhất, 16 nơ-ron ở lớp ẩn thứ hai, learning rate = 0,001, weight decay = 0,0001, batch size = 32 và 100 epoch, nhóm tiến hành đánh giá trên tập validation.

Kết quả thu được Validation RMSE = **1,0887** trên thang `log1p(SalePrice)`. Validation MAE sau khi chuyển dự đoán về đơn vị USD đạt **101.171,77 USD**, trong khi hệ số R^2 trên thang log đạt **-5,8515**.

Chỉ số	Kết quả
Validation RMSE (log1p)	1,0887
Validation MAE (USD)	\$101.171,77
Validation R ² (log1p)	-5,8515

Bảng 4.1. Kết quả đánh giá MLP baseline

Kết quả baseline cho thấy cấu hình ban đầu chưa đạt hiệu quả tốt trên tập validation. Đây là cơ sở để nhóm tiếp tục thực hiện tuning nhằm điều chỉnh kiến trúc và các siêu tham số của MLP.



Hình 4.4. Diễn biến Train Loss và Validation Loss của MLP baseline

Đường cong cho thấy loss giảm mạnh trong những epoch đầu và tiếp tục giảm chậm hơn ở các epoch sau. Validation loss có xu hướng dao động và không giảm tương ứng ổn định với train loss, cho thấy cấu hình baseline chưa đạt trạng thái tổng quát hóa tốt. Kết quả này thúc đẩy bước tuning ở phần tiếp theo.

4.3. Tuning mô hình MLP

4.3.1. Mục đích tuning

Hiệu năng của MLP phụ thuộc đáng kể vào việc lựa chọn các siêu tham số. Do đó, nhóm tiến hành tuning nhằm tìm cấu hình phù hợp hơn với dữ liệu Ames Housing. Các tham số được xem xét gồm:

- số lượng nơ-ron của lớp ẩn thứ nhất;
- số lượng nơ-ron của lớp ẩn thứ hai;
- learning rate;
- weight decay;

- batch size;
- số epoch;
- dropout.

Mỗi cấu hình được huấn luyện độc lập trên cùng tập training và được đánh giá bằng Validation RMSE trên cùng tập validation nhằm đảm bảo khả năng so sánh giữa các thí nghiệm.

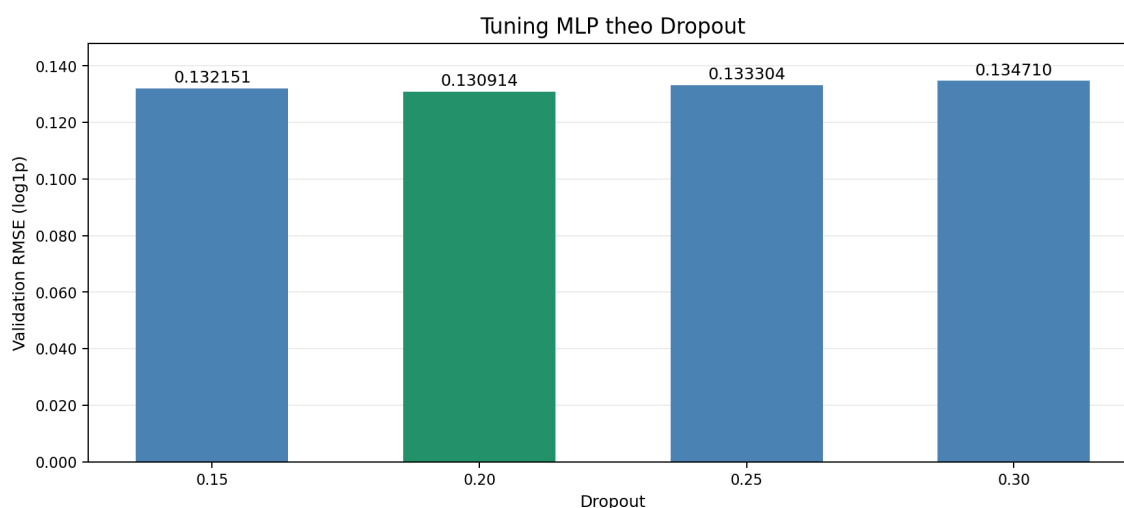
4.3.2. Thiết kế các cấu hình tuning

Dựa trên kết quả của mô hình baseline, nhóm tiến hành tìm kiếm trong một vùng siêu tham số mới. Kiến trúc được điều chỉnh từ 32–16 xuống 24–12 nơ-ron, đồng thời learning rate được tăng lên 0,00375 và weight decay được tăng lên 0,005. Batch size được giữ ở mức 32 và số epoch tăng lên 125.

Trong giai đoạn thử nghiệm này, nhóm tập trung đánh giá ảnh hưởng của Dropout với bốn giá trị lần lượt là 0,15; 0,20; 0,25 và 0,30. Các cấu hình được huấn luyện trên cùng tập training và đánh giá trên cùng tập validation nhằm bảo đảm khả năng so sánh.

Experiment	Hidden 1	Hidden 2	Learning rate	Weight decay	Batch	Epochs	Dropout	Val. RMSE
1	24	12	0,00375	0,005	32	125	0,15	0,132151
2	24	12	0,00375	0,005	32	125	0,20	0,130914
3	24	12	0,00375	0,005	32	125	0,25	0,133304
4	24	12	0,00375	0,005	32	125	0,30	0,134710

Bảng 4.2. Kết quả tuning MLP theo Dropout



Hình 4.5. So sánh Validation RMSE giữa các cấu hình MLP

Kết quả cho thấy cấu hình Experiment 2 với cấu hình được lựa chọn gồm hidden1 = 24, hidden2 = 12, learning rate = 0,00375, weight decay = 0,005, batch size = 32, epochs = 125 và Dropout = 0,20.

Đạt Validation RMSE thấp nhất, bằng 0,1309 trên thang $\log_{1p}(\text{SalePrice})$ và được sử dụng làm MLP tốt nhất cho các thí nghiệm tiếp theo. So với mô hình baseline có RMSE 1,0887, kết quả sau tuning giảm đáng kể, cho thấy việc điều chỉnh kiến trúc và siêu tham số đã cải thiện rõ rệt khả năng dự đoán của MLP.

4.3.3. Cấu hình MLP được lựa chọn

Từ kết quả tuning, nhóm lựa chọn cấu hình Experiment 2 làm MLP tốt nhất cho các thí nghiệm tiếp theo. Cấu hình được lựa chọn gồm 24 nơ-ron ở lớp ẩn thứ nhất, 12 nơ-ron ở lớp ẩn thứ hai, learning rate = 0,00375, weight decay = 0,005, batch size = 32, 125 epoch và Dropout = 0,20.

Cấu hình này đạt Validation RMSE = 0,130914, tương đương khoảng 0,1309 trên thang $\log_{1p}(\text{SalePrice})$. Vì mục tiêu của giai đoạn tiếp theo là tiếp tục kiểm tra khả năng cải thiện bằng các mô hình có cơ chế học khác nhau, MLP tuned được giữ làm mô hình cơ sở để so sánh với XGBoost, CatBoost và các mô hình tuyến tính regularized.

Thành phần	Giá trị
Input size	234
Hidden layer 1	24
Hidden layer 2	12
Activation	ReLU
Dropout	0,20
Optimizer	Adam
Learning rate	0,00375
Weight decay	0,005
Batch size	32
Epochs	125
Loss function	MSELoss

Bảng 4.4. Cấu hình MLP tốt nhất sau tuning

4.4 Thử nghiệm các mô hình bổ sung

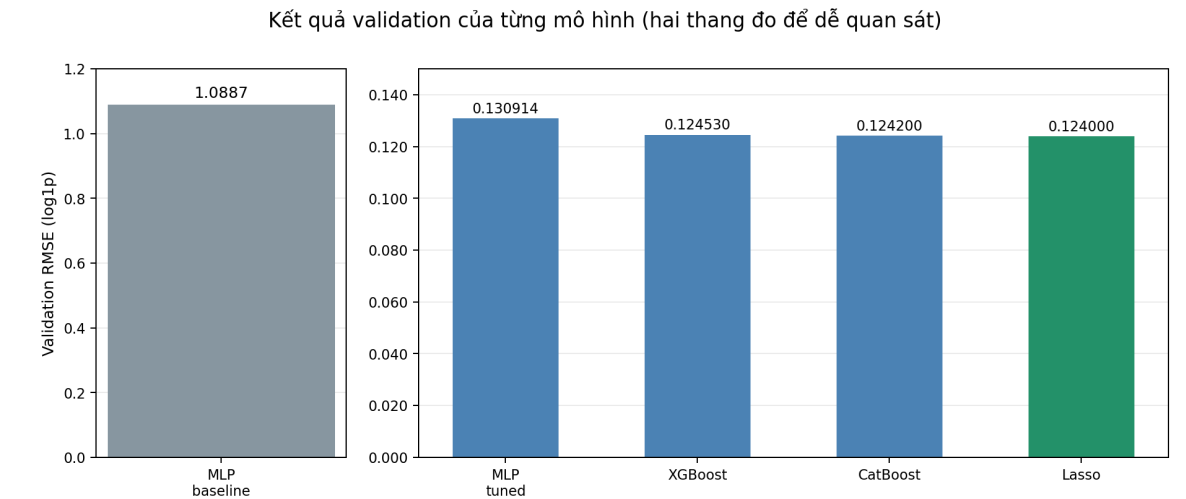
Sau khi có MLP tuned, nhóm thử XGBoost, CatBoost và các mô hình tuyến tính có regularization trên cùng phân chia train/validation. Các kết quả này giúp so sánh hiệu

quả của từng mô hình riêng lẻ và tìm những dự đoán có thể bổ sung cho nhau khi kết hợp.

Mô hình	Nhóm phương pháp	Validation RMSE
MLP baseline	Neural Network	1,0887
MLP tuned	Neural Network	0,130914
XGBoost	Gradient Boosting	0,124530
CatBoost	Gradient Boosting	0,1242
Lasso	Linear Regression + L1	0,1240
ElasticNet	Linear Regression + L1/L2	0,1239
Ridge	Linear Regression + L2	0,1252

Bảng 4.5. So sánh kết quả của các mô hình đơn lẻ

Mặc dù ElasticNet đạt Validation RMSE thấp nhất trong nhóm mô hình tuyến tính, Lasso lại cho kết quả tốt hơn khi được kết hợp với champion MLP–XGBoost. Điều này cho thấy hiệu năng của mô hình đơn lẻ không hoàn toàn quyết định mức đóng góp của mô hình trong ensemble.



Hình 4.6. So sánh Validation RMSE giữa các mô hình

Kết quả cho thấy các mô hình boosting và hồi quy tuyến tính regularized đều cải thiện rõ rệt so với MLP baseline. Tuy nhiên, mục tiêu của nhóm không chỉ là tìm mô hình đơn lẻ có RMSE nhỏ nhất mà còn kiểm tra khả năng bổ sung lẫn nhau giữa các mô hình. Vì vậy, các dự đoán được tiếp tục sử dụng trong các thí nghiệm ensemble.

Nhóm thực hiện thêm một nhánh fine-tuning XGBoost để khảo sát ảnh hưởng của các tham số độ sâu cây, sampling và regularization. Kết quả của nhánh này được dùng cho phân tích nội bộ và không được sử dụng trực tiếp trong công thức cuối cùng.

4.5 Xây dựng mô hình Ensemble

4.5.1 Cơ sở của Weighted ensemble

Đề tài dụng sự khác biệt trong cách học của các mô hình, nhóm kết hợp các dự đoán bằng weighted average. Nếu $\hat{y}_k$ là dự đoán của mô hình thứ k và w_k là trọng số tương ứng, dự đoán ensemble được tính theo:

$$\hat{y}_{ensemble} = \sum w_k \hat{y}_k, \quad \text{với } \sum w_k = 1$$

4.5.2. Từ MLP đến Ensemble ba mô hình

Nhóm trước tiên kết hợp MLP tuned và XGBoost bằng phương pháp weighted average. Cấu hình tốt nhất cho hai mô hình được xác định thông qua quá trình tìm kiếm trọng số trên tập validation.

Tiếp theo, nhóm bổ sung CatBoost nhằm kiểm tra khả năng cải thiện của một mô hình boosting khác và khai thác sự khác biệt trong dự đoán giữa các mô hình. Kết quả cho thấy CatBoost có thể đóng góp thêm thông tin khi kết hợp với MLP và XGBoost, mặc dù RMSE của CatBoost khi sử dụng riêng lẻ chưa phải thấp nhất.

Trong nhánh thử nghiệm ba mô hình, trọng số tốt nhất trên tập validation là:

MLP: 0,355

XGBoost: 0,355

CatBoost: 0,290

Nhánh MLP–XGBoost–CatBoost đạt Validation RMSE = 0,119519, thấp hơn tổ hợp MLP 0,38 và XGBoost 0,62 (0,119930) một lượng nhỏ. CatBoost được dùng trong nhánh so sánh này, không nằm trong tổ hợp tạo file nộp cuối.

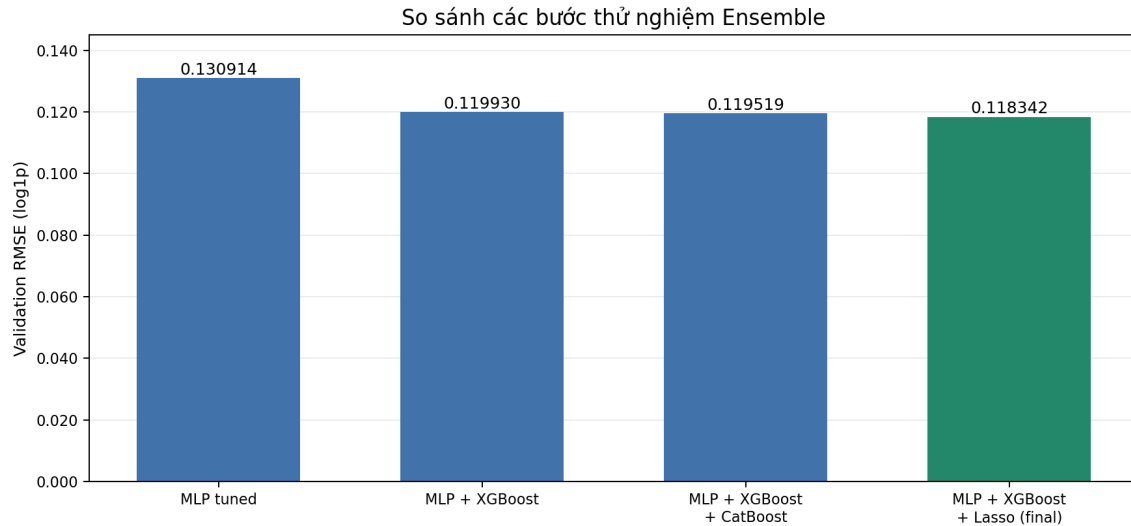
Kết quả cho thấy việc lựa chọn thành phần ensemble không chỉ dựa trên RMSE của từng mô hình riêng lẻ. Một mô hình có sai số riêng lẻ cao hơn vẫn có thể cải thiện kết quả chung nếu cung cấp các dự đoán bổ sung cho những mô hình còn lại.

4.5.3. Bổ sung Lasso vào Ensemble

Sau khi so sánh nhánh CatBoost, nhóm tiếp tục thử Ridge, Lasso và ElasticNet với champion gồm MLP 0,38 và XGBoost 0,62. Trong các mô hình tuyến tính, Lasso cho kết quả kết hợp tốt nhất với champion này trên tập validation.

Nhóm tìm kiếm trọng số trên tập validation và chọn Lasso 0,34 cùng champion MLP–XGBoost 0,66. Tổ hợp này đạt RMSE log1p = 0,118342.

- Ở bước tiếp theo, nhóm không tiếp tục kết hợp đồng thời CatBoost và các mô hình tuyến tính mà quay lại champion MLP–XGBoost để đánh giá riêng khả năng bổ sung của nhóm mô hình tuyến tính. Cách làm này giúp tách biệt ảnh hưởng của Lasso, Ridge và ElasticNet đối với champion cơ sở.



Hình 4.7. Quá trình cải thiện Validation RMSE qua các bước Ensemble

Từ trọng số champion MLP 0,38; XGBoost 0,62 và tỷ lệ champion/Lasso 0,66/0,34, trọng số cuối được tính như sau:

Giai đoạn	Trọng số	Validation RMSE
MLP tuned	—	0,130914
MLP + XGBoost	MLP 0,38; XGB 0,62	0,119930
MLP + XGB + CatBoost	MLP 0,355; XGB 0,355; CAT 0,290	0,119519
MLP + XGBoost + Lasso	MLP 0,2508; XGB 0,4092; Lasso 0,3400	0,118342

Bảng 4.6. Tiến trình cải thiện của mô hình Ensemble

Tổng các trọng số bằng 1. Tổ hợp cuối MLP–XGBoost–Lasso đạt Validation RMSE = 0,118342, thấp hơn nhánh thử nghiệm MLP–XGBoost–CatBoost (0,119519).

Kết quả cho thấy Lasso có khả năng bổ sung thông tin cho các mô hình neural network và tree boosting, dù RMSE của Lasso khi sử dụng riêng lẻ không phải thấp nhất.

4.6. Kiểm tra độ ổn định bằng 5-Fold Out-of-Fold (OOF)

4.6.1 Thiết lập OOF

Để kiểm tra mức độ ổn định của các mô hình trên toàn bộ 1.456 mẫu sau khi loại outlier, nhóm thực hiện 5-Fold Cross-Validation theo phương pháp Out-of-Fold. Trong mỗi fold, bốn phần dữ liệu được sử dụng để huấn luyện và phần còn lại được dùng để tạo prediction OOF. Đối với MLP, StandardScaler được fit riêng trên dữ liệu train của từng fold nhằm tránh rò rỉ thông tin từ fold validation.

4.6.2 Kết quả OOF

Mô hình	OOF RMSE
MLP	0,146882
XGBoost	0,119370
CatBoost	0,116351
Ensemble theo OOF weights	0,114819

Bảng 4.7. Kết quả 5-Fold OOF

Kết quả tìm kiếm trọng số trên prediction OOF cho thấy cấu hình MLP = 0,16, XGBoost = 0,20 và CatBoost = 0,64 đạt OOF RMSE = 0,114819. Đây là kết quả của thí nghiệm 5-Fold OOF trên toàn bộ tập dữ liệu huấn luyện và được sử dụng để kiểm tra khả năng tổng quát hóa của các mô hình.

Tuy nhiên, các trọng số OOF không được sử dụng trực tiếp trong công thức dự đoán cuối của notebook. Sau thí nghiệm OOF, nhóm quay lại pipeline validation chính và tiếp tục đánh giá các mô hình tuyến tính Ridge, Lasso và ElasticNet nhằm kiểm tra khả năng bổ sung của chúng vào champion MLP–XGBoost. Do đó, kết quả OOF được xem là một thí nghiệm kiểm chứng bổ sung, không phải cấu hình cuối cùng của bài toán.

4.7. Thử nghiệm mô hình tuyến tính và hoàn thiện Champion

4.7.1. So sánh Ridge, Lasso và ElasticNet

Sau thí nghiệm OOF, nhóm tiếp tục thử các mô hình tuyến tính có regularization trên cùng tập train/validation chính. Ridge đạt RMSE = 0,1252; Lasso đạt 0,1240; ElasticNet đạt 0,1239. Mặc dù ElasticNet có RMSE riêng lẻ thấp hơn Lasso một lượng rất nhỏ, Lasso lại tạo ra kết quả blend tốt nhất khi kết hợp với champion hiện tại.

4.7.2 Tinh chỉnh thông số Lasso

Nhóm quét trọng số Lasso từ 0,20 đến 0,60 với bước 0,01. Lần chạy dùng cho file nộp mới chọn Lasso 0,34 và nhánh MLP–XGBoost 0,66, đạt RMSE validation 0,118342 trên thang log1p.

Thành phần	Trọng số cuối
MLP	0,2508
XGBoost	0,4092

CatBoost	0,0000
Lasso	0,3400
Tổng	1,0000

Bảng 4.8. Trọng số Champion cuối cùng của giai đoạn Modeling

4.8. Tổng kết giai đoạn Modeling

Nhóm bắt đầu bằng MLP PyTorch, tiếp tục tuning bốn mức Dropout và thử XGBoost, CatBoost cùng các mô hình tuyến tính. Sau các nhánh ensemble thử nghiệm, công thức được dùng để tạo file nộp mới là MLP 0,2508; XGBoost 0,4092; Lasso 0,3400, có RMSE validation log1p 0,118342 trên 437 căn nhà.

Mô hình / cấu hình	Validation RMSE
MLP baseline	1,088700
MLP tuned	0,130914
XGBoost	0,124530
CatBoost	0,124200
Lasso	0,124000
MLP + XGBoost	0,119930
MLP + XGB + CatBoost	0,119519
MLP + XGBoost + Lasso	0,118342

Bảng 4.9. Tổng kết kết quả giai đoạn Modeling

Các giá trị validation của từng nhánh là kết quả nội bộ, còn Public Score Kaggle được tính trên dữ liệu kiểm tra ẩn. Phần Evaluation trình bày các độ đo của tổ hợp cuối và phần Deployment mô tả cách tạo CSV cùng kết quả hai lần nộp Kaggle.

5. ĐÁNH GIÁ MÔ HÌNH (EVALUATION)

5.1. Mục đích và dữ liệu đánh giá

Sau giai đoạn xây dựng mô hình, nhóm đánh giá dự đoán trên 437 căn nhà validation có giá bán thực tế. Từ 1.460 dòng train.csv, nhóm loại 4 dòng có GrLivArea trên 4.000 sq ft sau đó chia 1.456 dòng còn lại thành 1.019 mẫu huấn luyện và 437 mẫu validation. Các mô hình cơ sở được huấn luyện trên phần training; tập validation được dùng để so sánh mô hình và chọn trọng số kết hợp.

Tập test.csv gồm 1.459 căn nhà và không có SalePrice, vì vậy không thể tính RMSE hoặc MAE trực tiếp trên tập này. Nhóm tính các chỉ số trên 437 mẫu validation có nhãn, kết quả Kaggle trên dữ liệu kiểm tra ẩn được trình bày riêng ở phần triển khai.

5.2. Các độ đo sử dụng

Nhóm sử dụng RMSE (Root Mean Squared Error) làm chỉ số chính: lấy căn bậc hai của trung bình bình phương sai số giữa giá dự đoán và giá thực tế. Vì sai số được bình phương, một số dự đoán lệch nhiều sẽ ảnh hưởng mạnh đến RMSE.

Biến mục tiêu được chuyển thành $\log_{1p}(\text{SalePrice})$ trước khi huấn luyện. RMSE trên thang $\log_{1p}$ là chỉ số chính khi chọn cấu hình và gần với cách chấm điểm của Kaggle. Chỉ số này càng thấp càng tốt; nó không có đơn vị USD và không thể hiểu là phần trăm dự đoán đúng.

Để diễn giải sai số theo giá nhà, nhóm áp dụng `expm1` cho cả nhãn và dự đoán rồi tính MAE và RMSE theo USD. MAE là độ lệch tuyệt đối trung bình, còn RMSE theo USD nhạy hơn với các căn nhà có sai số lớn. Hai chỉ số này bổ sung cách nhìn trực quan cho ValidationRMSE trên thang log, được sử dụng làm chỉ số chính để so sánh mô hình.

5.3. Kết quả đánh giá mô hình cuối

MLP được thử với 4 mức Dropout (0,15; 0,20; 0,25; 0,30), các kết quả được lưu trong `tuning_results.csv`. Sau đó nhóm thử XGBoost, CatBoost, Ridge, Lasso và ElasticNet, đồng thời so sánh những phép kết hợp có trọng số trên tập validation. `best_tuned_model.pt` chỉ chứa trọng số MLP tốt nhất không chứa toàn bộ tổ hợp mô hình.

Trong bảng tuning MLP do nhóm lưu, thí nghiệm số 2 có kết quả tốt nhất: hai lớp ẩn 24 và 12 nơ-ron, learning rate 0,00375, weight decay 0,005, batch size 32, 125 epoch, Dropout 0,20. RMSE validation của MLP này xấp xỉ 0,1309 trên thang $\log_{1p}$.

Ở lần chạy notebook dùng để tạo file nộp mới, tổ hợp cuối cùng gồm MLP, XGBoost và Lasso với trọng số lần lượt 0,2508; 0,4092 và 0,3400. Trên 437 mẫu validation, mô hình đạt $\text{RMSE}_{\log_{1p}} = 0,118342$, $\text{MAE}_{\log_{1p}} = 0,077854$, $\text{MAE} = 12.917,53$ USD và $\text{RMSE} = 19.585,99$ USD. Các giá trị này được tính từ từng cặp giá thực tế và giá dự đoán, đồng thời lưu trong `evaluation_metrics.csv`.

5.4. Nhận xét và giới hạn

So với MLP tuned có RMSE khoảng 0,1309, tổ hợp ba mô hình cho RMSE validation thấp hơn. Tuy nhiên, các trọng số ensemble được chọn dựa trên chính tập validation này vì vậy mức 0,118342 có thể lạc quan hơn hiệu năng trên dữ liệu nhà chưa quan sát. Nhóm cũng kiểm tra 5-Fold Out-of-Fold cho MLP, XGBoost và CatBoost như một thí nghiệm bổ sung, nhưng đó không phải điểm Kaggle của tổ hợp cuối.

MAE 12.917,53 USD là sai lệch tuyệt đối trung bình trên tập validation, không phải mức sai số cố định cho mọi căn nhà. RMSE theo USD lớn hơn MAE cho thấy có những trường hợp lệch giá đáng kể. Ngoài ra, việc điền khuyết và mã hóa trên dữ liệu ghép train/test trước khi tách validation có thể khiến thống kê đặc trưng của validation đi vào khâu chuẩn bị dữ liệu, đây là giới hạn cần lưu ý khi diễn giải kết quả nội bộ.

6. TRIỂN KHAI VÀ NỘP BÀI (DEPLOYMENT)

6.1. Dự đoán trên tập test

Notebook tạo file nộp mới bằng tổ hợp MLP, XGBoost và Lasso. MLP tuned được chuyển sang chế độ đánh giá để tắt Dropout, XGBoost và Lasso sử dụng các mô hình đã fit

ở những cell trước. File `best_tuned_model.pt` là checkpoint của riêng MLP; để tái tạo dự đoán ensemble cần chạy notebook để có đủ cả ba thành phần.

Tập `test.csv` được biến đổi thành đúng 234 cột đặc trưng cùng thứ tự với dữ liệu huấn luyện, đầu vào MLP được chuẩn hóa bằng `StandardScaler` đã fit trên phần training. Mỗi thành phần tạo một dự đoán trên thang $\log_{1p}(\text{SalePrice})$. Notebook lấy trung bình có trọng số MLP 0,2508; XGBoost 0,4092; Lasso 0,3400 rồi dùng `expm1` để đổi kết quả cuối về USD.

Trước khi xuất file, chương trình kiểm tra có đủ 1.459 dự đoán, Id không trùng và giữ nguyên thứ tự của `test.csv` mọi giá dự đoán đều hữu hạn và lớn hơn 0. Nếu không thỏa điều kiện, notebook báo lỗi để nhóm kiểm tra trước khi nộp.

6.2. Tạo file submission

Kết quả được lưu trong `submission.csv` với đúng hai cột Id và SalePrice. Mỗi dòng chứa mã căn nhà và giá bán dự đoán khi ghi CSV, notebook dùng `index=False` để không tạo thêm cột chỉ số.

File nộp có 1.459 dòng dự đoán tương ứng với 1.459 dòng của `test.csv`, không có giá trị thiếu hoặc vô hạn. Nhóm lưu thêm `evaluation_metrics.csv` để đối chiếu kết quả validation và `tuning_results.csv` để theo dõi bốn thí nghiệm MLP. Checkpoint `best_tuned_model.pt` chỉ lưu thành phần MLP của tổ hợp cuối.

6.3. Nộp bài và diễn giải điểm Kaggle

Nhóm nộp file `submission.csv` lên cuộc thi *House Prices - Advanced Regression Techniques*. Kaggle chấp nhận file và hiển thị Public Score 0,12885. Trước đó, file `submission_ensemble_test.csv` của nhóm có điểm 0,12835; lần nộp mới cao hơn 0,00050, nghĩa là kém hơn một chút theo tiêu chí điểm càng thấp càng tốt. Điểm Kaggle là sai số trên thang log, không phải phần trăm dự đoán đúng hay sai số tính bằng USD.

RMSE validation 0,118342 và Public Score Kaggle 0,12885 được tính trên hai tập nhà khác nhau, nên không thể coi chúng là cùng một phép kiểm tra. Chênh lệch 0,010508 cho thấy kết quả trên tập chấm ẩn cao hơn sai số validation; điều này phù hợp với giới hạn của việc dùng cùng tập validation để chọn trọng số. Kết quả Kaggle xác nhận file đúng định dạng và có thể chấm, nhưng không chứng minh mô hình chính xác cho mọi giao dịch.

KẾT LUẬN

Nhóm đã triển khai bài toán dự đoán giá nhà theo sáu giai đoạn CRISP-DM: hiểu bài toán, hiểu dữ liệu, chuẩn bị dữ liệu, xây dựng mô hình, đánh giá và triển khai. MLP được xây dựng bằng PyTorch theo yêu cầu học phần sau khi thử thêm các mô hình bổ sung, notebook dùng tổ hợp MLP, XGBoost và Lasso để tạo 1.459 dự đoán. Lần nộp mới được Kaggle chấp nhận với Public Score 0,12885.

House Prices - Advanced Regression Techniques

Predict sales prices and practice feature engineering, RFs, and gradient boosting



Overview Data Code Models Discussion Leaderboard Rules Team Submissions

Submissions

All Successful Errors		Recent ▾
Submission and Description		Score ⓘ
✓	submission (1).csv Complete · PhamHuu · 1h ago · second try	0.12885
✓	submission_ensemble_test.csv Complete · SGHPK0905 · 1d ago	0.12835

Kết quả validation RMSE log1p 0,118342 và MAE 12.917,53 USD cho thấy tổ hợp có khả năng dự đoán trên dữ liệu giữ lại, dù vẫn có các căn nhà sai lệch nhiều. Điểm Kaggle của lần nộp mới cao hơn 0,00050 so với file submission_ensemble_test.csv đã có, vì vậy nhóm ghi nhận cả hai thay vì khẳng định lần nộp sau tốt hơn. Các hướng tiếp theo là kiểm tra tiền xử lý theo từng fold và xác nhận hiệu năng trên nhiều cách chia dữ liệu.

Link docs:

<https://docs.google.com/document/d/1517Y2y501hK7qkSjQFpFnCxeiQAoMKif3RUG4TdYYkM/edit?tab=t.0>

Link Github:

https://github.com/SGHPK0905/DeepLearning_3SV_DDU1231